asked to find only the Kth largest element. Are there other approaches we can consider to reduce this work?

Think about the quick sort technique, where we partition the array around a pivot element. After the partition step, the pivot element is in its correct and final position, while all the other elements are yet unsorted. But the catch here is that we can't find the pivot element in the Kth largest place. Therefore, we need a trial and error approach. We need to keep on trying pivot values such that we can hit upon the pivot element present at K-1st place in the final sorted array in descending order.

We can use the divide and conquer approach to reduce the work done in each step. Take a random element from the array, and partition the array around this element. Find the final resting index of your pivot element. Call it 'pivot_index'. If it is equal to (K-1), you have found the Kth largest element. If it is less than (K-1), you only need to consider the array starting from 'pivot_index + 1' for the Kth largest element. If the 'pivot index' is greater than (K-1), you need to consider only the array starting from 0 to 'pivot_index – 1'. Thus, you can end up reducing the work you do in each step, so that the array you need to examine for the Kth largest becomes smaller and smaller. In the average case, we can show that selecting the Kth largest element can be done in linear time. I will leave it to our readers to write the code for each of the above approaches as an exercise.

Now let us look at a few more coding questions that you can practice on.

1. There is a set of buildings on a street, standing from west to east. Any building that has a taller one on its west cannot have a view of the sunset. Can you find the buildings from which you can view the sunset? The building heights are given as a set of distinct integers in an array. What is the time and space complexity of your solution?
2. There is a set of N houses located on a street. The distance of each house from the start of the street is represented as a distinct integer and is provided in an array. You are asked to set up K mail boxes on the street, so that the total distance the residents from all houses need to walk to get their mails is minimised. Where would you place the K mail boxes? Let us think about some simplifying cases. If K is 1, where should you place the mail box? If K is N, where can you place the mail boxes? How can you generalise for any arbitrary value of K 1<=K<=N?
3. Consider an extension of the above problem. You are also given the number of residents for each house. This number can be varying widely for different houses, with values between 2 to 18. You are now asked to set up the K mail boxes such that the total distance walked by all residents from all houses is minimised. Where would you place the K mail boxes? Again, solve this problem first for K = 1 and then for any arbitrary value of K.
4. You are given a terabyte sized file of IP addresses. You need to find an IP address that is not present in the file. Each IP address is a 32-bit integer. Note that fitting the entire set of addresses into memory is not possible. What is the time and space complexity of your solution?
5. You are given a stream of integers coming in. At every new integer coming in, you need to compute the median for the current set of integers seen so far. One possibility is to record all the numbers seen so far, and add the current newly incoming number to that set. And use this stored set of numbers to compute the median. This requires you to process all the elements at every step, and is wasteful. How can you come up with an incremental algorithm that does not redo the whole computation at each step?
6. Often, instead of being given a completely unsorted array, you are given an almost sorted array. Basically, each element has possibly been disturbed from its correct position by utmost K elements. This is known as a k-sorted array. For example, 1, 5, 2, 8, 9 is a 2-sorted array. Can you come up with a solution to get this array fully sorted? What is the time and space complexity?
7. You are given an array A of N integers. You need to create an array B from elements of A such that B[0] <= B[1] >=B[2] <= B[3] >= B[4], and so on for N elements. Basically, the adjacent pairs of elements in B are sorted in opposite directions. How will you construct the array B from A? One possibility is to sort the entire array. Now divide the array into two halves, and interleave one element each from each half. Basically, you would end up with A[0], A[N/2+1], A[1], A[N/2+2], and so on. Alternatively, consider alternate pairs of the sorted array A and swap them — A[0], A[2], A[1], A[3], and so on. Both these approaches require O(nlogn) time complexity, since we end up sorting the entire array. Can you come up with a more efficient approach?

Feel free to reach out to me over LinkedIn/email if you need any help in your coding interview preparation. If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Wishing all our readers happy coding until next month! Stay healthy and stay safe. END

By: Sandya Mannarswamy

The author is an expert in natural language processing and is currently working as an independent researcher. Her interests include natural language processing, machine learning and AI.